

TASK 04 — Asynchronous Communication with Apache Kafka

Step 1 — Study messaging concepts

Message

A **message** is a piece of information sent from one application/service to another.

Example:

Order ID: 101

Customer: Madhav

Amount: ₹5,000

Status: CREATED

In Kafka, this information is sent as a Kafka record/message.

Real-world example

When a customer places an order:

Order Service

| "Order 101 created"

Kafka

The message can contain information such as:

```
{  
  "orderId": 101,  
  "customerName": "Madhav",  
  "amount": 5000,  
  "status": "CREATED"  
}
```

Event

An **event** represents something that **has happened**.

Example:

OrderCreated

PaymentCompleted

PaymentFailed

UserRegistered

ProductUpdated

OrderCancelled

An event usually describes a change or action that occurred.

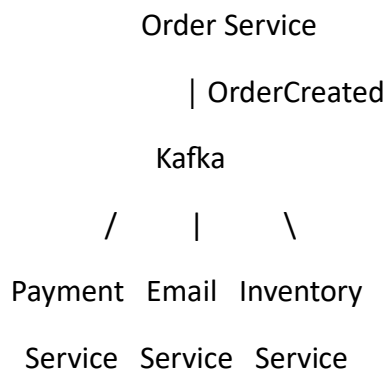
Example

A customer places an order.

The Order Service generates:

OrderCreated

Kafka can distribute this event to other services.



This is one of the major advantages of **event-driven architecture**.

Producer

A **Producer** is an application or service that **sends messages/events to Kafka**.

Example:

Order Service

| sends OrderCreated

Kafka

Here:

Order Service = Producer

Simple definition

Producer = Sender

Example, a Spring Boot Order Service may publish:

OrderCreated

to a Kafka topic.

Consumer

A **Consumer** is an application or service that **reads messages/events from Kafka**.

Example:

Kafka

|

Payment Service

Here:

Payment Service = Consumer

The Payment Service consumes:

OrderCreated

and starts payment processing.

Simple definition

Consumer = Receiver/Reader

Topic

A **Topic** is a named stream/category where Kafka stores messages.

Think of a topic like a **folder or channel** for related events.

Example:

orders

payments

users

notifications

You could have:

orders topic

OrderCreated

OrderCreated

OrderCancelled

OrderUpdated

A producer sends messages to a topic:

Order Service

|

orders topic

Consumers subscribe to that topic:

orders topic

|—————▶ Payment Service

|—————▶ Inventory Service

|—————▶ Notification Service

A topic is **not normally a single message**.

It is a stream of many records.

Partition

A Kafka topic can be divided into multiple **partitions**.

For example:

orders topic

Partition 0

Message 0

Message 1

Message 2

Partition 1

Message 3

Message 4

Message 5

Partition 2

Message 6

Message 7

Message 8

Partitions allow Kafka to process large amounts of data efficiently.

Why partitions:

Suppose you have millions of orders.

Instead of putting everything into one sequence:

orders

|— Order 1

|— Order 2

|— Order 3

|— ...

|— Order 1,000,000

Kafka can distribute the records:

orders topic

/ | \

Partition Partition Partition

0 1 2

This enables **parallel processing**.

Messages within a particular partition maintain their ordering.

example:

Partition 0:

OrderCreated

OrderPaid

OrderShipped

OrderDelivered

The order is preserved within that partition.

Consumer Group

A **Consumer Group** is a group of consumers that work together to consume messages from Kafka.

Suppose a topic has 3 partitions:

orders topic

Partition 0

Partition 1

Partition 2

And we have three consumers in one group:

Consumer Group: OrderGroup

Consumer 1 —► Partition 0

Consumer 2 —► Partition 1

Consumer 3 —► Partition 2

They can process the data in parallel.

Important rule

Within a consumer group, **one partition is assigned to only one consumer at a time.**

For example:

Partition 0 —► Consumer A

Partition 1 —► Consumer B

Partition 2 —► Consumer C

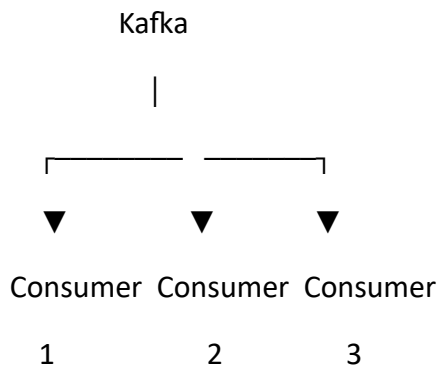
This prevents the same partition's records from being processed by multiple consumers in the same group simultaneously.

Why Consumer Groups are useful

Suppose your application receives 1 million messages.

One consumer may be too slow.

You can add more consumers:



This provides **parallel processing and scalability.**

Offset

An **offset** is the position of a message within a Kafka partition.

Example:

Partition 0

Offset	Message
-----	-----
0	Order 101
1	Order 102
2	Order 103
3	Order 104
4	Order 105

Here:

Order 101 → offset 0

Order 102 → offset 1

Order 103 → offset 2

Kafka consumers use offsets to keep track of **which messages they have processed**.

Why is offset important:

Suppose the consumer processed:

Offset 0

Offset 1

Offset 2

and then crashes.

When it restarts, Kafka can use the consumer group's stored offset to determine where processing should continue.

Processed:

0 ✓

1 ✓

2 ✓

3 ← next processing position

4

5

This helps Kafka provide reliable message processing.

Complete Kafka Flow

Suppose a customer place an order.

Step 1 — Producer creates an event

Order Service

creates:

```
{  
  "orderId": 101,  
  "status": "CREATED"  
}
```

Step 2 — Producer sends it to Kafka

Order Service

| OrderCreated

Kafka

Step 3 — Kafka stores it in a topic

orders topic

|— Partition 0

|— Partition 1

└— Partition 2

Step 4 — Consumer reads it

Kafka

|

Payment Service

Step 5 — Consumer processes it

Payment Service

|

Process Payment

Step 6 — Offset is tracked

Partition 0

Offset 0 → Order 101 ✓

Offset 1 → Order 102 ✓

Offset 2 → Order 103 ← current

Step 2 — Synchronous vs Asynchronous Communication

Synchronous Communication

Here, **Order Service directly calls Notification Service and waits for a response.**

Example

Suppose a customer places an order.

The Order Service needs to send a notification:

Order Service

| HTTP Request

Notification Service

| Process notification

Response

|

Order Service

The Order Service is waiting while the Notification Service processes the request.

Real-world example

Customer

|

Order Service

| "Send order confirmation"

Notification Service

| "Notification sent"

Order Service

Only after receiving the response does the Order Service continue.

Typical technology

Synchronous communication is commonly implemented using:

- REST API
- HTTP
- OpenFeign
- gRPC

Example:

Order Service → REST API → Notification Service

Problem with Synchronous Communication

Suppose the Notification Service is slow or unavailable.

Order Service

| Request

Notification Service

X

Not available

The Order Service may have to wait for a timeout.

Example:

Order Service

| HTTP request

Notification Service

| 30 seconds...

Timeout

This creates **tight coupling** between the two services.

Problems

- Order Service depends directly on Notification Service.
- If Notification Service is down, communication fails.
- The caller may have to wait.
- Network failures can affect the request.
- High traffic can increase waiting time.

Asynchronous Communication

Here, the Order Service **does not directly call** the Notification Service.

Instead, it sends an event to Kafka.

Order Service

| OrderCreated event

Kafka

|

Notification Service

The Notification Service reads the event from Kafka and processes it later.

How Asynchronous Communication Works

Let's use an online shopping example.

Step 1 — Customer places an order

Customer



Order Service

Order Service creates:

OrderCreated

Step 2 — Order Service publishes event

Order Service

| OrderCreated

Kafka

Kafka stores the event in a topic, for example:

orders

Step 3 — Order Service continues

This is the important difference.

The Order Service does **not need to wait** for Notification Service.

Order Service

| Publish event

Kafka

└───▶ Order Service continues

Step 4 — Notification Service consumes the event

Kafka

| OrderCreated

Notification Service

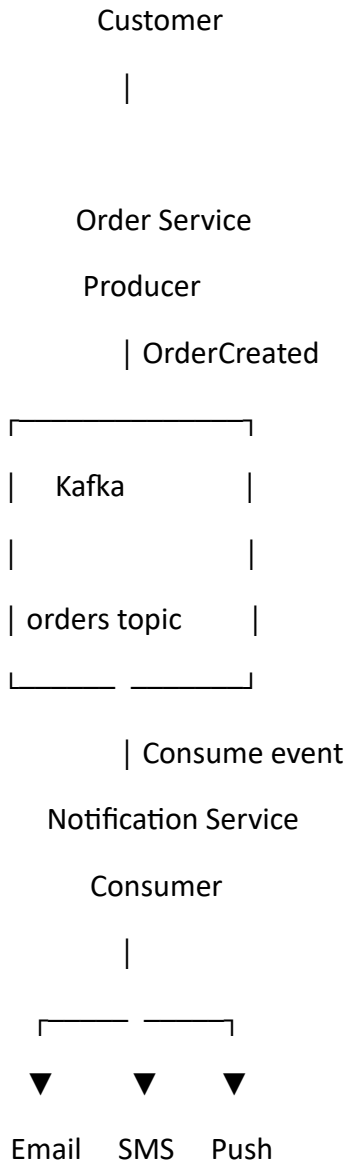
The Notification Service receives the event and sends:

Email

SMS

Push Notification

5. Complete Asynchronous Flow



6. Main Difference

Synchronous

Direct communication

Caller waits

Request → Response

More tightly coupled

Asynchronous

Communication through Kafka/message broker

Caller doesn't need to wait

Event → Processing

More loosely coupled

Synchronous

Usually HTTP/REST

Failure can immediately affect caller

Good for immediate responses

Asynchronous

Kafka/events/messages

Consumer can process when available

Good for background/event processing

7. Simple Example

Synchronous

Imagine you order food and call the restaurant.

You



Restaurant



"Your order is accepted"

You are directly communicating with the restaurant and waiting for its response.

Asynchronous

Instead, imagine placing the order into a queue:

You



Queue



Restaurant

You don't need to wait while the restaurant prepares the order.

Step 3 — Set Up Kafka

Set up Kafka — Run Kafka locally using an appropriate development setup and verify that Kafka is running.

For your Spring Boot microservices project, the overall learning path is:

Step 1

Kafka concepts



Step 2

Synchronous vs Asynchronous



Step 3

Install & run Kafka



Step 4

Create Kafka Topic



Step 5

Build Producer



Step 6

Build Consumer



Step 7

Connect Spring Boot services



Step 8

Test OrderCreated event

Synchronous: Order Service → Notification Service → Response

Asynchronous: Order Service → Kafka → Notification Service

In the asynchronous approach, **Kafka acts as the communication layer between the services**, allowing the producer and consumer to work independently.

Step 4 — Create Topic

Create:

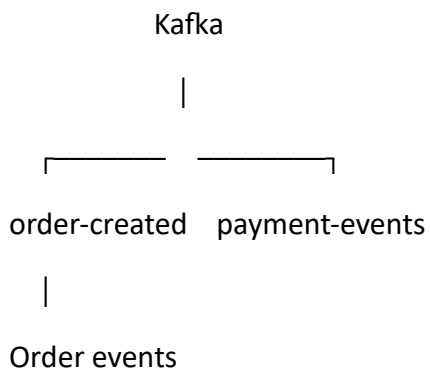
order-created

What is order-created:

order-created is the **Kafka topic name**.

A topic is where Kafka stores the events/messages sent by producers.

Think of it like a communication channel:



For this project, we create:

order-created

The **Order Service** will publish order events to this topic.

The **Notification Service** will consume events from this topic.

Why do we need a topic:

Without a topic, the producer and consumer don't have a specific Kafka stream to communicate through.

The flow becomes:

Order Service

| publish

order-created topic

| consume

Notification Service

Topic = A named stream/channel in Kafka where related messages are stored.

Step 5 — Create Producer

Order Service publishes:

And gives this JSON:

```
{  
  "orderId": 501,  
  "userId": 101,  
  "amount": 50000,  
  "eventType": "ORDER_CREATED"  
}
```

Here, the **Order Service is the Producer.**

What does the Producer do:

The producer sends this event to Kafka.

Order Service

| JSON Event

Kafka

The event is:

```
{  
  "orderId": 501,  
  "userId": 101,  
  "amount": 50000,  
  "eventType": "ORDER_CREATED"  
}
```

Understanding the JSON

Let's understand each field.

orderId

"orderId": 501

This identifies the order.

Here:

Order ID = 501

userId

"userId": 101

This identifies the customer/user who created the order.

User ID = 101

amount

"amount": 50000

This represents the order amount.

Amount = ₹50,000

eventType

"eventType": "ORDER_CREATED"

This tells the consumer **what happened**.

In this case:

ORDER_CREATED

means:

A new order has been created.

Complete Producer Flow

When the customer creates order 501:

Customer

| Create Order

Order Service

| Creates event

{

"orderId": 501,

"userId": 101,

"amount": 50000,

"eventType": "ORDER_CREATED"

}

| publish

Kafka

|

order-created

So: Order Service = Producer

and:

order-created = Topic

Step 6 — Create Consumer

This means the **Notification Service acts as a Kafka Consumer**.

The Order Service is the **Producer**.

Kafka is between them.

Roles

Component	Role
Order Service	Producer
Kafka	Message broker

Component	Role
-----------	------

order-created	Topic
---------------	-------

Notification Service Consumer

notification-group	Consumer Group
--------------------	----------------

Add Kafka Dependency

If you are using Maven, add this to pom.xml.

```
<dependency>
  <groupId>org.springframework.kafka</groupId>
  <artifactId>spring-kafka</artifactId>
</dependency>
```

A basic pom.xml can look like:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.5</version>
    <relativePath/>
  </parent>
  <groupId>com.example</groupId>
  <artifactId>notification-service</artifactId>
  <version>0.0.1-SNAPSHOT</version>
```

```
<properties>
  <java.version>17</java.version>
</properties>
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.kafka</groupId>
    <artifactId>spring-kafka</artifactId>
  </dependency>
</dependencies>
<build>
  <plugins>
    <plugin>
      <groupId>
        org.springframework.boot
      </groupId>
      <artifactId>
        spring-boot-maven-plugin
      </artifactId>
    </plugin>
  </plugins> </build>

</project>
```

Configure Kafka

Create: application.yml

inside: src/main/resources/

Add:

server:

port: 8084

spring:

application:

name: notification-service

kafka:

bootstrap-servers: localhost:9092

consumer:

group-id: notification-group

auto-offset-reset: earliest

key-deserializer: org.apache.kafka.common.serialization.StringDeserializer

value-deserializer: org.apache.kafka.common.serialization.StringDeserializer

producer:

key-serializer: org.apache.kafka.common.serialization.StringSerializer

value-serializer: org.apache.kafka.common.serialization.StringSerializer

Important properties

bootstrap-servers: localhost:9092

This tells the application where Kafka is running.

group-id: notification-group

This identifies the consumer group.

auto-offset-reset: earliest

If the consumer has no previous offset, Kafka starts reading from the earliest available message.

Main Application

Create: NotificationServiceApplication.java

Code:

```
package com.example.notification;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class NotificationServiceApplication {

    public static void main(String[] args) {

        SpringApplication.run(

            NotificationServiceApplication.class,

            args

        );

    }

}
```

Create Event DTO

Suppose the Order Service publishes:

```
{

    "orderId": 101,

    "userId": 501,

    "productName": "Laptop",

    "amount": 75000

}
```

Create:

OrderCreatedEvent.java

```
package com.example.notification.dto;
```

```
public class OrderCreatedEvent {  
    private Long orderId;  
    private Long userId;  
    private String productName;  
    private Double amount;  
    public OrderCreatedEvent() {  
    }  
    public OrderCreatedEvent(  
        Long orderId,  
        Long userId,  
        String productName,  
        Double amount) {  
        this.orderId = orderId;  
        this.userId = userId;  
        this.productName = productName;  
        this.amount = amount;  
    }  
    public Long getOrderId() {  
        return orderId;  
    }  
    public void setOrderId(Long orderId) {  
        this.orderId = orderId;  
    }  
    public Long getUserId() {  
        return userId;  
    }  
}
```

```
public void setUserId(Long userId) {  
    this.userId = userId;  
}  
public String getProductName() {  
    return productName;  
}  
public void setProductName(String productName) {  
    this.productName = productName;  
}  
public Double getAmount() {  
    return amount;  
}  
public void setAmount(Double amount) {  
    this.amount = amount;  
}  
@Override  
public String toString() {  
    return "OrderCreatedEvent{" +  
        "orderId=" + orderId +  
        ", userId=" + userId +  
        ", productName='" + productName + "\" +  
        ", amount=" + amount +  
        "'}";  
}  
}
```

Create Kafka Consumer

Now create: OrderConsumer.java

```
package com.example.notification.consumer;

import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.stereotype.Component;

@Component

public class OrderConsumer {

    @KafkaListener(
        topics = "order-created",
        groupId = "notification-group"
    )

    public void consumeOrder(String message) {

        System.out.println(
            "Received Order Event: " + message
        );
    }
}
```

Step 7 — Process Event

Create Notification Service

Create: NotificationService.java

```
package com.example.notification.service;

import org.springframework.stereotype.Service;

@Service

public class NotificationService {

    public void processOrderCreated(String message) {

        System.out.println(
```

```

        "Order created event received."

    );

    System.out.println( "User notification required."

);

    System.out.println( "Event: " + message);

}

}

```

Update Consumer

Now inject NotificationService.

```

package com.example.notification.consumer;

import com.example.notification.service.NotificationService;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.stereotype.Component;

@Component
public class OrderConsumer {

    private final NotificationService notificationService;

    public OrderConsumer(

        NotificationService notificationService) {

        this.notificationService = notificationService;

    }

    @KafkaListener(

        topics = "order-created",

        groupId = "notification-group"

    )

    public void consumeOrder(String message) {

        System.out.println(

```

```

        "Received event from Kafka: " + message
    );
    notificationService.processOrderCreated(message);
}
}

```

Now the flow is:

Kafka

|

OrderConsumer

|

NotificationService

|

Process notification

Better Version Using JSON

The previous version receives JSON as a String.

For example:

```

{
  "orderId": 101,
  "userId": 501,
  "productName": "Laptop",
  "amount": 75000
}

```

We can convert this JSON into:

OrderCreatedEvent

Use Jackson.

Spring Boot already provides Jackson through its web dependencies, but you can explicitly add:

```
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-databind</artifactId>
</dependency>
```

Then:

```
package com.example.notification.consumer;

import com.example.notification.dto.OrderCreatedEvent;
import com.example.notification.service.NotificationService;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.stereotype.Component;

@Component
public class OrderConsumer {

    private final ObjectMapper objectMapper;
    private final NotificationService notificationService;

    public OrderConsumer(
        ObjectMapper objectMapper,
        NotificationService notificationService) {
        this.objectMapper = objectMapper;
        this.notificationService = notificationService;
    }

    @KafkaListener(
        topics = "order-created",
        groupId = "notification-group"
    )
    public void consumeOrder(String message) {
```

```

try {
    OrderCreatedEvent event =
        objectMapper.readValue(
            message,
            OrderCreatedEvent.class
        );

    System.out.println("Order ID: " + event.getOrderId());
    System.out.println("User ID: " + event.getUserId());
    notificationService.processOrderCreated(message);
} catch (Exception e) {
    System.err.println("Failed to process event: " + e.getMessage());
}
}
}

```

Process Notification

Update: NotificationService.java

```

package com.example.notification.service;

import com.example.notification.dto.OrderCreatedEvent;
import org.springframework.stereotype.Service;

@Service

public class NotificationService {

    public void sendOrderNotification(
        OrderCreatedEvent event) {

        System.out.println("=====");
        System.out.println("ORDER CREATED");
        System.out.println("Order ID: " + event.getOrderId());
    }
}

```

```

        System.out.println("User ID: " + event.getUserId());
        System.out.println("Product: " + event.getProductName());
        System.out.println("Amount: " + event.getAmount());
        System.out.println("User notification required");
        System.out.println("=====");
    }
}

```

Then consumer:

```

package com.example.notification.consumer;

import com.example.notification.dto.OrderCreatedEvent;
import com.example.notification.service.NotificationService;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.stereotype.Component;

@Component
public class OrderConsumer {

    private final ObjectMapper objectMapper;
    private final NotificationService notificationService;

    public OrderConsumer(
        ObjectMapper objectMapper,
        NotificationService notificationService) {

        this.objectMapper = objectMapper;
        this.notificationService = notificationService;
    }

    @KafkaListener(
        topics = "order-created",

```

```

        groupId = "notification-group"
    )
    public void consumeOrder(String message) {
        try {
            OrderCreatedEvent event =
                objectMapper.readValue(
                    message,
                    OrderCreatedEvent.class
                );
            notificationService
                .sendOrderNotification(event);
        } catch (Exception e) {
            System.err.println(
                "Error processing Kafka event: "
                + e.getMessage()
            );
        }
    }
}

```

Step 8 — Test Event Publishing

Start Kafka

If you're using Docker, a simple modern Kafka setup can be run with Docker Compose.

For example:

services:

kafka:

image: apache/kafka:latest

container_name: kafka

ports:

- "9092:9092"

Start:

docker compose up -d

Check:

docker ps

You should see Kafka running.

Create Kafka Topic

Create:

order-created

For Apache Kafka container setups, the exact command can vary by image/version. With the Apache Kafka image, you can use:

```
docker exec -it kafka \  
/opt/kafka/bin/kafka-topics.sh \  
--create \  
--topic order-created \  
--bootstrap-server localhost:9092
```

Check topics:

```
docker exec -it kafka \  
/opt/kafka/bin/kafka-topics.sh \  
--list \  
--bootstrap-server localhost:9092
```

You should see:

order-created

Publish Test Event

You can manually publish a message.

```
docker exec -it kafka \  
/opt/kafka/bin/kafka-console-producer.sh \  
--topic order-created \  
--bootstrap-server localhost:9092
```

Then enter:

```
{"orderId":101,"userId":501,"productName":"Laptop","amount":75000}
```

Press:

Enter

Kafka now contains the event.

Step 9 — Test Consumer

This step tests the **consumer side** of Kafka.

What is happening:

The complete flow is:

Order Service

| 1. Create Order

Kafka Topic

| 2. Event stored

Notification Service

| 3. Consume Event

Process Notification

For example, the Order Service creates:

Order ID = 101

User ID = 501

Product = Laptop

The Order Service publishes an event:

OrderCreated

Kafka receives this event.

The Notification Service is listening to the Kafka topic, so it receives the event.

should verify

There are three things to verify.

1. Kafka received the event

You should confirm that the event was successfully published to the Kafka topic.

Order Service



Kafka



OrderCreated Event

2. Notification Service received the event

The Notification Service should receive the event from Kafka.

Kafka



Notification Service

You can verify this through the application logs.

For example:

Received OrderCreated event

Order ID: 101

User ID: 501

3. Notification Service processed the event

Receiving the message is not enough.

The Notification Service must actually process it.

For example:

OrderCreated event received



Read order information



Identify user



Prepare notification



Send/process notification

Step 10 — Test Failure

Create an order.

Then:

Restart Notification Service.

Finally:

Verify that the architecture handles the pending event according to the configured consumer behavior.

This step tests what happens when the **consumer is temporarily unavailable**.

Stop Notification Service

First, stop the Notification Service.

Your architecture now looks like:

Order Service



Kafka



X

Notification Service

|

STOPPED

Kafka is still running.

The Order Service is still running.

Only the Notification Service is stopped.

Create an Order

Now create a new order.

For example:

Order ID: 102

User ID: 502

Product: Mobile Phone

The Order Service creates the order and publishes an event.

Order Service

|

OrderCreated Event

|

Kafka

The important point is that the Notification Service is currently stopped.

So there is nobody consuming the event at that moment.

What happens to the event:

Kafka is designed to store records in topics.

Conceptually:

Kafka Topic

|— Event 100

|— Event 101

└─ Event 102 ← New event

The Notification Service is offline.

Therefore, the event may remain available for the consumer according to the topic's retention settings and the consumer group's committed offset.

This is why Kafka is useful for asynchronous communication.

Restart Notification Service

Now start the Notification Service again.

The architecture becomes:

Order Service

|

Kafka

|

Notification Service

|

Process Event

The Notification Service reconnects to Kafka.

Kafka determines what records the consumer group still needs to consume based on its offsets and configured consumer behavior.

The pending event can then be delivered to the Notification Service.

Verify the Pending Event

Suppose the order created while Notification Service was stopped was:

Order ID: 102

After restarting the Notification Service, check the logs.

You may see:

Received OrderCreated event

Order ID: 102

User ID: 502

Processing notification...

This demonstrates that the system can handle a temporary consumer outage.

What Does "Configured Consumer Behavior" Mean:

"Verify that the architecture handles the pending event according to the configured consumer behavior."

This means Kafka's behavior depends on configuration such as:

Consumer Group

For example:

notification-group

This identifies the group of consumers processing the topic.

Consumer Offset

Kafka tracks where a consumer group has reached in the topic.

Conceptually:

Event 1 → Event 2 → Event 3 → Event 4

↑

Consumer

position

If the consumer has not successfully processed a particular event, the consumer's position and commit behavior determine what happens when it reconnects.

Auto Offset Reset

A common configuration is:

earliest

This tells Kafka what starting position to use when there is **no existing committed offset** for that consumer group.

Important: earliest does **not** mean "always reread every message whenever the service restarts." Existing committed offsets still matter.